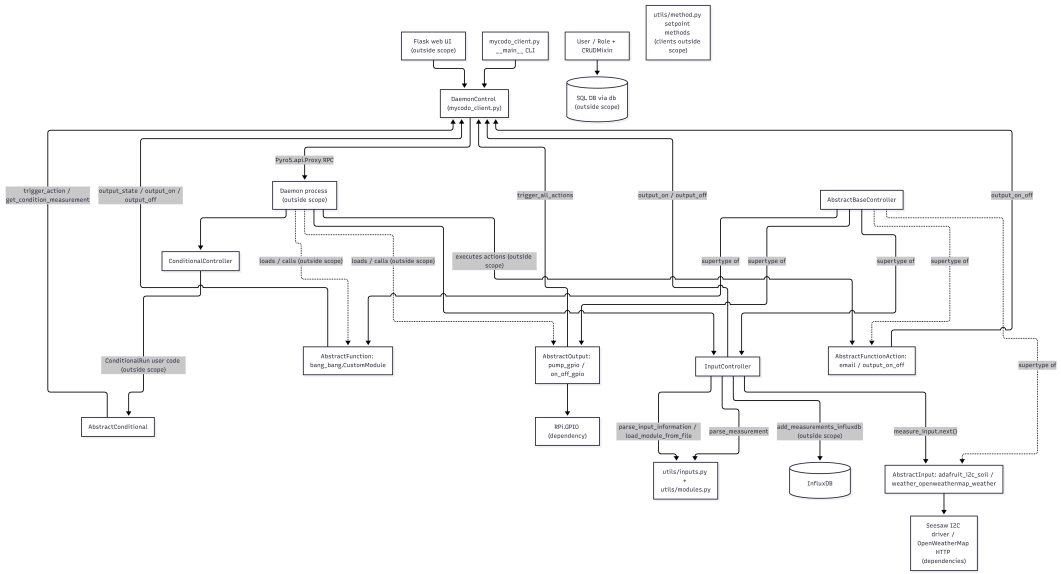
Design Patterns in Mycodo

23CSE455 Design Patterns case study · Team B10 · Agriculture management

AM.SC.U4AIE23038	Johan Rajesh	AIE
AM.SC.U4AIE23003	Aadarsh Jayan Nair	AIE
AM.SC.U4AIE23029	Dano Aby Alex	AIE
AM.SC.U4AIE23140	P Arya	AIE

Application: Mycodo as an agriculture manager

- Mycodo v8.17.0 (Python / Flask, GPL-3.0), commit `ee70a111`
- Daemon of controller threads + web UI; every device module extends `AbstractBaseController`
- Scope: 24 files — soil & weather inputs, pump outputs, bang-bang irrigation, setpoint schedules, alerts, RPC client, users
- Flow: InputController → AbstractInput.read → soil driver → InfluxDB → bang-bang → DaemonControl → pump



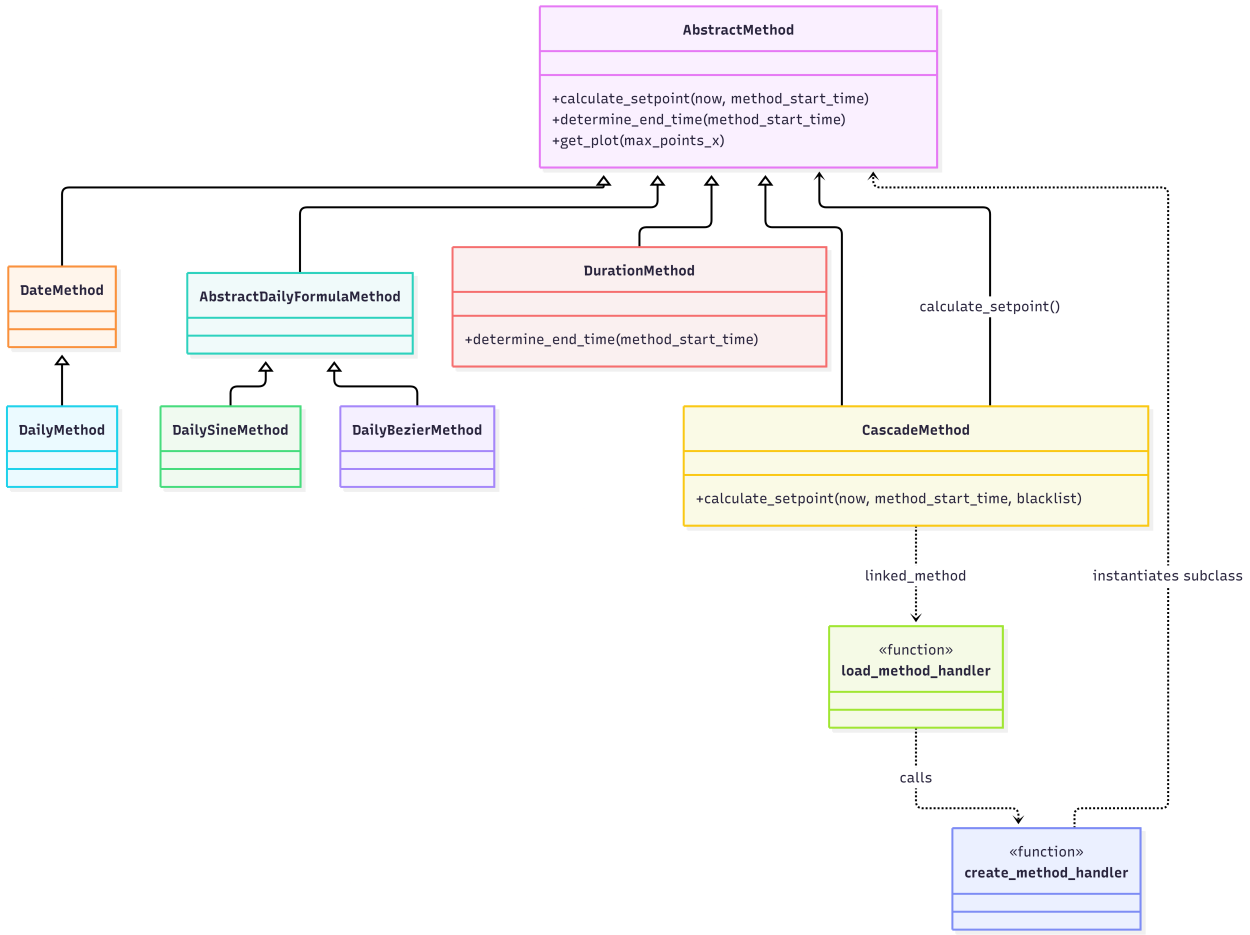
Part 1 – Pattern discovery at the baseline

Pattern	Category	#	Instances
Template Method	GoF Behavioral	6	Sensor Measurement Acquisition; Module Initialization with Retry; Controller Thread Lifecycle; Output Actuation (irrigation switching); Formula Setpoint Plot Sampling; Date/Daily Setpoint Interpolation
Strategy	GoF Behavioral	1	Setpoint Schedule Calculation
Adapter	GoF Structural	1	Sensor Driver Adaptation
Proxy	GoF Structural	1	Daemon RPC Client
Command	GoF Behavioral	1	Automation/Alert Actions
Plugin	Architectural	1	Input/Output Module Loading
Layer Supertype	Architectural	1	Daemon Module Base
Active Record	Architectural	1	User and Role Persistence

8 types
13 instances

- 18 concrete participant classes, 85 unique symbols
- 23 rejected, 5 uncertain candidates (e.g. Observer for output triggers, Composite for cascades)
- Every role checked against `data/code/`; 112 line ranges verified by script

Example: Strategy – Setpoint Schedule Calculation



`create_method_handler` picks a setpoint algorithm by type; `CascadeMethod` combines other strategies through the same interface.

Part 2 – Three agriculture changes

Change	Smell → refactoring	Feature	Pattern effect	Tests
CH01 Crop types	Duplicated Code → Extract Method	Crop profiles (tomato, rice, wheat) set target moisture; new crops registered in one place	2 modified, 11 preserved	25 → 42 pass
CH02 Irrigation strategies	Switch Statements → Replace Conditional with Polymorphism	Hysteresis, time-window, skip-if-rain policies; stackable	+ Strategy, + Decorator	48 → 64 pass
CH03 Sensor alerts	Long Method → Extract Method	Critically-dry / missing-sensor alerts to log and e-mail, once per episode	+ Observer	72 → 83 pass

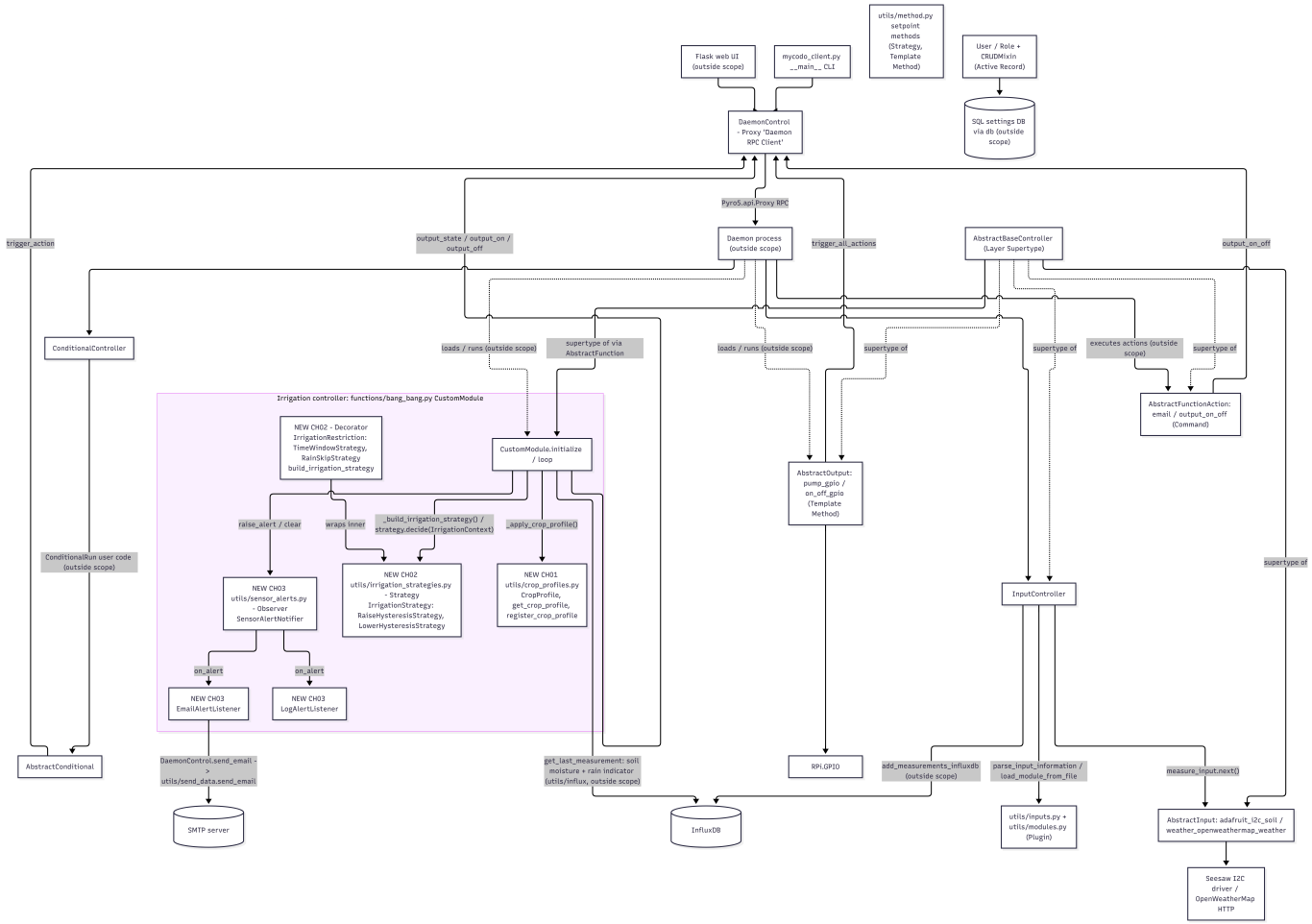
Each change = 3 commits: characterization tests → refactoring (tests unchanged) → feature with acceptance tests. Patches in `Part2/changes/commits/`.

Part 2 – Pattern counts and metrics per snapshot

Snapshot	Types	Instances	NLOC	Avg CCN	Tests
Original code	8	13	4024	3.6	25/25
Code after Change1	8	13	4098	3.5	42/42
Code after Change2	9	15	4298	3.5	64/64
Code after Change3	10	16	4426	3.3	83/83

- `bang_bang.loop()` cyclomatic complexity 14 → 14 → 7 → 5
- No instance removed or replaced; continuing collaborations keep their Part 1 names
- Trade-off: +10 % NLOC and more classes, but each policy / listener is small and tested alone

Part 2 – Architecture after CH03



Part 3 – Same problem in four languages (LOC / max CC)

Pattern	Java	Python	JavaScript	C++	Tests (each)
Template Method	77 / 7	37 / 6	54 / 11	85 / 21	12/12
Strategy	106 / 9	67 / 10	83 / 9	97 / 12	12/12
Adapter	108 / 7	80 / 7	88 / 7	106 / 7	12/12
Proxy	111 / 7	60 / 6	69 / 5	103 / 7	12/12
Command	112 / 8	67 / 7	88 / 9	130 / 8	12/12
Plugin	87 / 9	61 / 8	75 / 9	94 / 8	9/9
Layer Supertype	68 / 23	56 / 19	65 / 22	82 / 25	11/11
Active Record	113 / 14	69 / 13	83 / 7	119 / 14	12/12

32/32 runners pass on shared CSV test cases (9–12 cases each). LOC = lizard NLOC of the production file; CC = highest function CCN.

Part 3 – What changes across languages

- **Contract checks move:** Java / C++ reject a missing role method at compile time; Python / JavaScript at run time
- **Dynamic features shorten patterns:** Proxy via `__getattr__` / ES `Proxy` (60 / 69 NLOC) vs explicit interfaces (111 / 103)
- **Static typing costs size, not complexity:** max CC usually within a few points
- **Library gaps drive CC:** hand-written date parsing gives CC 21 in C++ vs 6 in Python

Language	Mean LOC	Median ms (range)
Java	98	156–235
Python	62	65–86
JavaScript	76	89–97
C++	102	31–34

Whole-process time: compile first, 1 warmup + 5 runs, median. Mostly runtime start-up, not pattern speed.

LLM use and human verification

- Claude Opus 5.5 in Claude Code; one recorded run per part (2026-10-11 / 10-12); every member wrote prompts per part
- Raw prompts, inputs and outputs kept in `Part*/llm/`; each claim graded in `llm.csv` (correct / partial / cannot confirm)
- Checks: every CSV line range verified by script; CSVs regenerated from raw tool output byte-for-byte; all tests re-run 2026-10-25
- Corrections: mislabelled Part 1 sub-totals, oversize Java / C++ snippets, lizard skipping `.hpp` files (fixed and re-measured)

Findings and limitations

- Mycodo's extensibility rests on Template Method around `AbstractBaseController` plus Plugin loading
- Agriculture changes fitted the structure: 13 → 16 instances, 8 → 10 types, nothing removed
- Patterns keep their intent in all four languages; only the enforcement point moves
- Limits: static analysis of 24 files; daemon / Command invoker out of scope; Part 2 tests use mocks; timings are start-up dominated